Compilerdiagnostik, Testing und Verifikation

Felix Orosz David Stargala

Seminar Compilerbau, Bachelor Informatik, 5. Semester
Wintersemester 2026/2027 Stand: 1. November 2026

Zusammenfassung

Die Ausarbeitung behandelt Compilerdiagnostik, Source Locations, Tests für Lexer, Parser und Codegenerator, Golden Tests, Differential Testing und die Prüfung erzeugten Maschinencodes. Beispiele aus LLVM, Clang, Rust, Python, Csmith und ESP-IDF erläutern den Testaufbau und seine Aussagegrenzen. Die Grundlagen der vorhergehenden Seminarthemen werden vorausgesetzt.

Inhaltsverzeichnis

1 Gute Compiler-Fehlermeldungen **2**

2 Source Locations **2**

3 Unit-Tests für Lexer, Parser und Codegenerator **4**

4 Golden Tests und FileCheck **6**

5 Differential Testing **8**

6 Erzeugten Maschinencode auf dem ESP32-C3 testen **10**

A Ergänzende Fundstellen **12**

Literatur **13**

1 Gute Compiler-Fehlermeldungen

Hilfreiche Compilerfehlermeldungen erleichtern es, einen Fehler zu lokalisieren und zu beheben. Die folgende Tabelle enthält illustrative Beispiele; der genaue Wortlaut hängt unter anderem von der Compilerversion und dem Quellcode ab.

Tabelle 1: Illustrative Beispiele für Compilerfehlermeldungen

Angabe	**Beispiel**
Schweregrad	`Error/Warn/Info/Critical`
Fehlerart	`Syntax Error`
Kontext	`In der Funktion main()`
Betroffene Position	`main.py, Zeile 10, Spalte 3`
Betroffener Quelltext	`print 1`
Vorschlag zur Behebung	`Füge Klammern hinzu`

Tests können prüfen, ob Compilerdiagnosen die erwarteten Meldungen und Positionsangaben enthalten. Ein Beispiel dafür sind die UI-Tests von Rust.

1.1 Praxisbeispiel: Rust

Die Sammlung `tests/ui` von Rust prüft unter anderem die Konsolenausgaben des Compilers. Mit *Error Annotations* werden erwartete Diagnosen direkt im Quelltext vermerkt. Die offizielle Dokumentation zeigt folgendes Beispiel [1]:

```
fn main() {
  boom  //~ ERROR cannot find value `boom` in this scope [E0425]
}
```

Listing 1: Originalbeispiel für eine Rust-Fehlerannotation

Das Tilde-Symbol (`~`) kennzeichnet die von einem Fehler betroffene Zeile. Die Annotation `~^^^` verweist auf eine Stelle drei Zeilen darüber. Zusätzlich werden generierte `.stderr`-Dateien genutzt, welche die erwartete Diagnoseausgabe für den Vergleich speichern. Auf die Positionsangabe folgen die Diagnosekategorie, etwa `ERROR`, `WARN`, `NOTE` oder `HELP`, und der erwartete Meldungstext [1].

2 Source Locations

2.1 Aufgabe und benötigte Informationen

Eine *Source Location* bezeichnet eine Stelle im Quelltext. Sie erlaubt es, Tokens und Knoten des *Abstract Syntax Tree* (AST) ihrer ursprünglichen Position im Quelltext zuzuordnen. Für eine spätere Diagnose müssen sich die betroffene Quelldatei und die Stelle innerhalb dieser bestimmen lassen, weshalb Positionsinformationen bereits beim Einlesen der Tokens erfasst werden [2], Abschnitte *The Token class* und *How to add an expression or statement.*

Eine *Source Range* beschreibt einen zusammenhängenden Ausschnitt des Quelltexts, etwa einen Variablennamen oder einen Ausdruck. Dafür eignen sich Anfangsposition und Länge oder zwei Bereichsgrenzen. Die Bedeutung der Endposition muss eindeutig sein, da sie das letzte enthaltene Zeichen oder die Position unmittelbar hinter dem Bereich bezeichnen kann. Wird dagegen der Anfang des letzten Tokens gespeichert, ist zusätzlich dessen Länge erforderlich [2, 3].

Damit die Informationen nutzbar bleiben, muss die Zuordnung zur ursprünglichen Eingabe erhalten bleiben. Da ein Variablenname mehrfach in derselben Datei verwendet werden kann, reicht der Name allein nicht aus, um die betroffene Stelle im Quelltext eindeutig zu bestimmen. Die in Tokens und AST gespeicherten Positionen ermöglichen es deshalb, erkannte Typ- oder Namensfehler der betroffenen Stelle im Quelltext zuzuordnen [2].

2.2 Beispiel: Source Locations und Source Ranges in Clang

Clang verwendet kleine Positionsobjekte der Klasse `SourceLocation`. Ein Token enthält seine Anfangsposition und die Länge seiner originalen Schreibweise im `SourceBuffer`. Der Zugriff auf den ursprünglichen Text bleibt damit möglich, ohne die Schreibweise rekonstruieren zu müssen [2], Abschnitte *The SourceLocation and SourceManager classes* und *The Token class*.

Bei einer *Token Range* bezeichnen die Grenzen jeweils den Anfang des ersten und des letzten Tokens [2]:

```
x = foo + bar;
^first    ^last
```

Listing 2: Originalbeispiel für eine Token Range

Die gespeicherte Endgrenze `last` liegt beim Zeichen `b` am Anfang des letzten Tokens `bar`. Zur Markierung des gesamten letzten Tokens muss sein Ende ermittelt werden. Dafür kann seine Länge mit `Lexer::MeasureTokenLength()` oder die Endposition mit `Lexer::getLocForEndOfToken()` ermittelt werden [2].

Am Wort `bar` lassen sich die Konventionen für die Endposition veranschaulichen. Bei einer Zählung der ASCII-Bytes ab 0 belegt `bar` die Positionen 10 bis 12. Mit eingeschlossener Endposition wird der Ausschnitt als `[10,12]` beschrieben. Bei ausgeschlossener Endposition lautet die Angabe `[10,13)`, wobei Position 13 unmittelbar hinter dem letzten Zeichen liegt und nicht dazugehört. Beide Angaben erfassen denselben Ausschnitt. Bei der Erzeugung und Verarbeitung einer Source Range muss dieselbe Konvention für die Endposition gelten.

2.3 Weitergabe und Darstellung

Der Lexer ordnet Tokens ihrer Schreibweise zu. Der Parser erhält diese Tokens und übernimmt die für Namen und Ausdrücke benötigten Positionen in den AST. Dadurch lassen sich Diagnosen aus späteren Analysen der ursprünglichen Stelle im Quelltext zuordnen [2].

Dabei wird die Position für die Ausgabe einer Diagnose anders dargestellt als bei ihrer Speicherung in Tokens und AST. Für die Ausgabe müssen die zugehörige Zeile und Spalte bestimmt werden. Die Zeile ergibt sich aus den vorhergehenden Zeilenumbrüchen. Der in Bytes gemessene Abstand vom Zeilenanfang entspricht jedoch nicht immer der sichtbaren Spalte: UTF-8 verwendet ein bis vier Bytes pro Unicode-Codepoint [4, Abschnitt 3]. Auch Tabulatoren können in der Darstellung mehrere Spalten einnehmen. Wird dies nicht berücksichtigt, können korrekt gespeicherte Positionen zu einer falschen Unterstreichung führen.

Für die Implementierung müssen daher Dateizuordnung, Positionsdarstellung und die Grenzen der Source Ranges konsistent aufeinander abgestimmt sein. Testfälle an Zeilenumbrüchen und am Dateiende untersuchen diese Zuordnung. Bei Zeichen, die in UTF-8 mehrere Bytes benötigen, wird zusätzlich geprüft, ob Bytepositionen bei der Darstellung korrekt in sichtbare Spalten umgesetzt werden. Die konkrete Datenstruktur kann je nach Compilerarchitektur unterschiedlich aufgebaut sein, wobei die beschriebenen Anforderungen unabhängig von der verwendeten Klassenbibliothek gelten [2, 4].

3 Unit-Tests für Lexer, Parser und Codegenerator

Unit-Tests prüfen einzelne Funktionen oder Komponenten einer Software [5]. Tests für eine bestimmte Compilerphase helfen, die Ursache eines Fehlers einzugrenzen.

Daneben werden phasenübergreifende Tests und End-to-End-Tests eingesetzt, die den gesamten Compilerzyklus abdecken.

3.1 Lexer

Der Lexer zerlegt Zeichenfolgen in Tokens. Bei Sprachen mit einem *Preprocessing*, also einer Vorverarbeitung, ist die Tokenverarbeitung mit dieser Phase verknüpft [2].

Fehlerquellen in dieser Phase sind falsch erkannte oder eingeordnete Whitespace-Zeichen, Zeilenumbrüche, Kommentare und Escape-Sequenzen [6].

3.1.1 Beispiel: Clang im LLVM-Projekt

```
EXPECT_TRUE(Lex("#include <\\\\\\n").empty());
```

Listing 3: Originalprüfzeile für eine unvollständige Include-Direktive

Der Ausschnitt aus den Clang-Unit-Tests im LLVM-Projekt überprüft die Verarbeitung einer unvollständigen `#include`-Direktive mit Backslashes und einem Zeilenumbruch. Dabei wird mit `EXPECT_TRUE` sichergestellt, dass die Funktion `Lex` für diese Eingabe eine leere Token-Liste zurückgibt [7].

Ein weiterer Test prüft, ob der Anfang von Bezeichnern korrekt gefunden wird:

```
TEST_F(LexerTest, GetBeginningOfTokenWithEscapedNewLine) {
  ...
  std::string TextToLex = "rabarbar\n"
                          "foo\\\nbar\n"
                          "foo\\\rbar\n"
                          "fo\\\r\nbar\n"
                          "foo\\\n\rba\n";
  std::vector<tok::TokenKind> ExpectedTokens{5, tok::identifier};
  std::vector<Token> LexedTokens = CheckLex(TextToLex, ExpectedTokens);

  for (const Token &Tok : LexedTokens) {
    FileIDAndOffset OriginalLocation =
        SourceMgr.getDecomposedLoc(Tok.getLocation());
    for (unsigned Offset = 0; Offset < IdentifierLength; ++Offset) {
      SourceLocation LookupLocation =
          Tok.getLocation().getLocWithOffset(Offset);

      FileIDAndOffset FoundLocation = SourceMgr.getDecomposedExpansionLoc(
          Lexer::GetBeginningOfToken(LookupLocation, SourceMgr, LangOpts));

      // Check that location returned by the GetBeginningOfToken
      // is the same as original token location reported by Lexer.
      EXPECT_EQ(FoundLocation.second, OriginalLocation.second);
    }
  }
}
```

Listing 4: Gekürzter Clang-Test für Tokenpositionen bei Zeilenumbrüchen

Der Test [7] prüft, ob der Lexer von jeder Zeichenposition eines Wortes dessen Anfang findet. Das soll auch dann funktionieren, wenn das Wort durch einen Backslash und einen Zeilenumbruch unterbrochen ist. Dafür werden verschiedene Arten von Zeilenumbrüchen getestet.

3.1.2 Beispiel: Python

Auch in interpretierten Sprachen wie Python werden Tokenizer-Tests durchgeführt [8]:

```
tokens5 = list(tokenize.tokenize(readline))
tokens2 = [tok[:2] for tok in tokens5]

bytes_from2 = tokenize.untokenize(tokens2)
readline2 = iter(bytes_from2.splitlines(keepends=True)).__next__
tokens2_from2 = [tok[:2] for tok in tokenize.tokenize(readline2)]

self.assertEqual(tokens2_from2, tokens2)
```

Listing 5: Auszug aus dem Python-Test zur erneuten Tokenisierung

Der Test zerlegt Quelltext in Tokens und berücksichtigt anschließend nur Tokentyp und Zeichenfolge. Die übrigen Felder von `TokenInfo` gehen nicht in den Vergleich ein [9]:

```
TokenInfo(type=%s, string=%r, start=%r, end=%r, line=%r)
```

Listing 6: Darstellung der Felder von TokenInfo

Die Positionen der Tokens sind für diesen Vergleich nicht relevant.

Anschließend setzt `untokenize` die vereinfachten Tokens wieder zu Quelltext zusammen. Dessen erneute Tokenisierung muss dieselben Paare aus Tokentyp und Zeichenfolge liefern wie vor der Rückkonvertierung [8].

3.2 Parser

Der Parser überführt die vom Lexer erzeugten Tokens in eine abstrakte Syntaxstruktur, in der Regel einen *Abstract Syntax Tree* (AST).

Dabei prüft er die syntaktischen Zusammenhänge zwischen den Tokens und bildet sie in der Struktur ab. Eine tiefere semantische Analyse oder Typprüfung wird jedoch erst in späteren Phasen durchgeführt [2].

```
let mut async = 1; //~ ERROR expected identifier, found keyword `async`
let mut r#async = 1; // OK
```

Listing 7: Auszug aus einem Rust-UI-Test für Schlüsselwörter

Der Ausschnitt aus den Rust-UI-Tests kennzeichnet die problematische Zeile mit der erwarteten Fehlermeldung [10, 1]. Der Test erfasst mehrere Compilerphasen; die Unterscheidung zwischen Schlüsselwörtern und Bezeichnern betrifft dabei den Parser.

Die erste Zeile ist fehlerhaft, denn das Schlüsselwort `async` darf nicht als Bezeichner verwendet werden. Ein Positivbeispiel ist die zweite Zeile, in welcher das Schlüsselwort `async` mit einem sogenannten *Raw Identifier*, also ausdrücklich als Bezeichner verwendet wird. Diese Funktionalität ist primär für Abwärtskompatibilität gedacht.

3.3 Codegenerator

Der Codegenerator übersetzt die interne Repräsentation in Code für die Zielplattform, beispielsweise in Assemblytext.

LLVM testet die Codegenerierung für unterschiedliche Zielplattformen beziehungsweise Architekturen, darunter RISC-V [11]:

```
define i32 @add(i32 %a, i32 %b) nounwind {
; RV32I-LABEL: add:
; RV32I:       # %bb.0:
; RV32I-NEXT:    add a0, a0, a1
; RV32I-NEXT:    ret
;
; RV64I-LABEL: add:
; RV64I:       # %bb.0:
; RV64I-NEXT:    addw a0, a0, a1
; RV64I-NEXT:    ret
  %1 = add i32 %a, %b
  ret i32 %1
}
```

Listing 8: LLVM-Additionsfunktion mit RV32I- und RV64I-Prüfmustern

Die Funktion in dieser *Intermediate Representation* (IR) addiert zwei 32-Bit-Zahlen und gibt das Ergebnis zurück. Die Kommentare mit `RV32I` und `RV64I` kennzeichnen die erwarteten Assembler-Ausgaben für die jeweiligen Zielplattformen. Dadurch können für einen IR-Codeblock zwei unterschiedliche Assemblyausgaben getestet werden.

Die Kommentare enthalten Prüfmuster für *FileCheck*, ein LLVM-Werkzeug zum automatischen Vergleich textbasierter Ausgaben. Abschnitt 4 erläutert die Prüfregeln [12]. Die erwarteten Instruktionen unterscheiden sich: Auf der 32-Bit-Plattform wird die Anweisung `add` verwendet, während auf der 64-Bit-Plattform die Anweisung `addw` verwendet wird.

4 Golden Tests und FileCheck

4.1 Vergleich mit einer geprüften Referenz

Ein Golden Test vergleicht eine aktuelle Ausgabe mit einer zuvor fachlich überprüften, gespeicherten Referenz, dem *Golden*. Ein solcher Vergleich kann sich auf Compiler-Ausgaben wie Diagnosen oder AST-Ausgaben sowie auf die Ausgabe eines übersetzten Programms beziehen. Entscheidend sind die festgelegte Eingabe, die geprüfte Referenz und der Umfang des Vergleichs [2, 13].

Ein vollständiger Referenzvergleich lässt sich in vier Schritten beschreiben:

1) Die Eingabe und das fachlich erwartete Ergebnis werden festgelegt.
2) Die Komponente beziehungsweise das übersetzte Programm wird ausgeführt.
3) Die tatsächliche Ausgabe wird erfasst und nach den festgelegten Vergleichsregeln mit der Referenz verglichen.
4) Stimmen die relevanten Daten überein, besteht der Test. Andernfalls wird die Abweichung untersucht.

Auch die LLVM-Testsammlung vergleicht Ausgaben ihrer Testprogramme mit gespeicherten Referenzen [13].

Die Referenz enthält lediglich die Erwartung. Wird eine falsche Ausgabe als Golden gespeichert, kann eine übereinstimmende falsche Ausgabe den Test bestehen. Die fachliche Prüfung der Referenz ist deshalb eine Voraussetzung für die Aussagekraft des Vergleichs.

4.2 Beispiel: C-Programm und LLVM-Referenzdatei

Das Programm `2002-05-02-CastTest1.c` und seine Datei mit der Endung `.reference_output` stehen im selben Ordner der LLVM-Testsammlung. Die folgenden beiden Listings geben ihren vollständigen Inhalt wieder [14]:

```
int printf(const char *, ...);

int
main(int argc, char** argv)
{
  signed char c0  = -1;
  unsigned char c1 = 255;
  printf("bs0  = %d %d\n", c0, c1);
  return 0;
}
```

Listing 9: C-Originalprogramm aus LLVM

```
bs0  = -1 255
exit 0
```

Listing 10: Referenzdatei

Die Variable `c0` ist vorzeichenbehaftet und enthält -1, `c1` ist vorzeichenlos und enthält 255. Auf dem hier betrachteten Ziel mit 8-Bit-`char` und 32-Bit-`int` passen beide Werte in `int`. Beim variadischen Aufruf von `printf` werden sie entsprechend den *Integer Promotions* zu `int` umgewandelt, wobei ihre Werte erhalten bleiben. Daher werden die Zahlen -1 und 255 ausgegeben [15].

Die erste Referenzzeile enthält die erwartete Textausgabe einschließlich der Leerzeichen. Die zweite Zeile beschreibt den Exit-Status des Testlaufs. `exit 0` wird nicht durch `printf` ausgegeben. Das Programm beendet `main` mit `return 0`, und die Referenz erfasst diesen Status zusätzlich [14].

Für die Durchführung werden somit Programmtext, beobachtete Ausgabe und Exit-Status benötigt. Ein Vergleich muss beispielsweise scheitern, wenn die zweite Zahl als -1 statt 255 erscheint oder das Programm mit einem anderen Status endet. Er bestätigt bei Erfolg das erwartete Verhalten dieses Testprogramms unter der verwendeten Umgebung, nicht die allgemeine Korrektheit des Compilers [14, 13].

4.3 Abweichungen und Pflege der Referenz

Eine Abweichung zeigt zunächst, dass die aktuelle Ausgabe die gespeicherte Erwartung nicht erfüllt. Bei unverändertem Programm und unveränderter Sprachsemantik bleibt das fachlich erwartete Ergebnis bestehen. Bei einer beabsichtigten Änderung der Ausgabe kann eine Anpassung ihrer Referenz notwendig sein. Zuvor muss geprüft werden, ob die neue Ausgabe dem erwarteten Verhalten entspricht. Wird die Referenz ungeprüft durch die aktuelle Ausgabe ersetzt, kann dadurch auch eine fehlerhafte Ausgabe zum neuen Sollwert werden [13].

Unterschiede der Umgebung können einen Textvergleich beeinflussen. Zeilenumbrüche können unterschiedlich gespeichert sein, beispielsweise als `CRLF` oder `LF`. Vor dem Vergleich können diese in ein gemeinsames Format umgewandelt werden. Diese Vereinheitlichung muss im Testaufbau berücksichtigt werden und darf keine Informationen entfernen, die der Test prüfen soll.

4.4 Beispiel: RV32I-Patterns mit FileCheck

FileCheck ermöglicht es, ausgewählte Patterns in der Compiler-Ausgabe zu prüfen. Dadurch kann der Vergleich auf bestimmte Instruktionen und ihre Reihenfolge beschränkt werden. Änderungen in anderen Ausgabeteilen müssen dadurch nicht zwingend zum Scheitern des Tests führen [12].

Die Prüfzeilen der Additionsfunktion aus Abschnitt 3.3 verwenden die Präfixe `RV32I` und `RV64I`, um Erwartungen für die beiden Zielplattformen zu unterscheiden [11].

FileCheck sucht zunächst das Funktionslabel `add:` und danach die Kommentarzeile `# %bb.0:`. Die beiden `RV32I-NEXT`-Direktiven verlangen unmittelbar anschließend die Addition `add a0, a0, a1` und die Rückkehr mit `ret`. Fehlt eine erwartete Textstelle oder ist diese Reihenfolge verletzt, schlägt die Prüfung fehl [12, 11].

Geprüft wird der erzeugte Assemblytext. Ob der Code bei seiner Ausführung das erwartete Ergebnis liefert, wird separat in Abschnitt 6 behandelt.

5 Differential Testing

Differential Testing vergleicht das beobachtbare Verhalten desselben Quellcodes bei unterschiedlichen Compilern oder Übersetzungskonfigurationen. Unter den folgenden Voraussetzungen sollten die Ergebnisse übereinstimmen.

Dabei werden folgende Varianten unterschieden:

- Cross-Compiler Testing [16]
- Cross-Version Testing [16]
- Cross-Optimization Testing [16]
- Cross-Architecture Testing [17]

5.1 Abweichungen

Eine wichtige Voraussetzung ist, dass der Quellcode kein undefiniertes oder unspezifiziertes Verhalten enthält (oder eben dieses kontrolliert testet) und relevante Unterschiede zwischen den Zielplattformen berücksichtigt werden.

In C und C++ sind Überläufe vorzeichenbehafteter Ganzzahlen und ganzzahlige Divisionen durch null Beispiele für undefiniertes Verhalten [18]. Unspezifiziertes Verhalten beschreibt unter anderem die Reihenfolge von Funktionsaufrufen, wenn diese keine Abhängigkeiten zueinander haben [19]. Während unspezifiziertes Verhalten bewusst mehr Flexibilität erlaubt, kann undefiniertes Verhalten zu unvorhersehbaren Ergebnissen führen [20].

5.2 Cross-Compiler Testing

Unterschiedliche Compiler, wie beispielsweise GCC und Clang, arbeiten intern unterschiedlich, müssen aber die Semantik eines Programms bei der Übersetzung erhalten.

5.3 Cross-Version Testing

Verschiedene Versionen eines Compilers werden miteinander verglichen, um Regressionen aufzudecken. Bei unveränderten Sprachregeln und vergleichbaren Compileroptionen sollte das beobachtbare Programmverhalten erhalten bleiben.

5.4 Cross-Optimization Testing

Compiler bieten unterschiedliche Optimierungsstufen an, beispielsweise `-O1`, `-O2` und `-O3`. Dabei können unter anderem Kontrollstrukturen verändert und Ausdrücke vereinfacht werden.

Die GCC-Option `-ffast-math` erlaubt Abweichungen von strengen Regeln für mathematische Operationen zugunsten schnellerer Ausführung. Programme, die auf deren genaue Einhaltung angewiesen sind, können dadurch andere Ergebnisse liefern [21]. Für solche Vergleiche können festgelegte Toleranzbereiche verwendet werden.

5.5 Cross-Architecture Testing

Das Programmverhalten wird auf unterschiedlichen Zielarchitekturen verglichen, beispielsweise x86, ARM, MIPS und RISC-V. Dabei müssen plattformabhängige Eigenschaften wie die Größe von Datentypen und die Byte-Reihenfolge berücksichtigt werden.

Die Größe von `long` hängt vom *Application Binary Interface* (ABI) ab. Unter x86-64 mit LP64 ist der Typ 64 Bit breit und hat den kleinsten Wert -9223372036854775808. Unter ARM32 mit ILP32 ist er 32 Bit breit; sein kleinster Wert beträgt -2147483648. Auch unterschiedliche Byte-Reihenfolgen (Endianness), Fließkommaarithmetik und sonstige plattformabhängige Eigenschaften können zu Abweichungen führen [22]. [23] [15]

Die Tests müssen solche Abweichungen dokumentieren und berücksichtigen. Dafür können sie für jede Zielarchitektur eigene erwartete Ergebnisse festlegen.

5.6 Beispiel: Csmith

Abweichende Ergebnisse können auf Compilerfehler hinweisen. Der Vergleich allein zeigt jedoch weder eindeutig, *welche* Konfiguration fehlerhaft ist, noch, *warum* der Fehler auftritt.

Csmith [24] generiert zufällige Programme in C. Diese können mit verschiedenen Compilern und Compileroptionen getestet werden. Die Ergebnisse werden anschließend verglichen, um mögliche Fehler aufzudecken.

```
csmith > random2.c
gcc random2.c -I$HOME/csmith/include -o random2_gcc
clang random2.c -I$HOME/csmith/include -o random2_clang
./random2_gcc > gcc_output.txt
./random2_clang > clang_output.txt
diff -u gcc_output.txt clang_output.txt
```

Listing 11: GCC-/Clang-Vergleich mit Csmith in einer Unix-Shell

Der obige Ausschnitt zeigt Befehle für eine Unix-Shell. Ein zufälliges C-Programm wird generiert, mit GCC und Clang kompiliert und anschließend ausgeführt. Die Ausgaben werden in Textdateien gespeichert und mit `diff` verglichen. Der Include-Pfad muss zur lokalen Csmith-Installation passen.

Die erzeugten Binärdateien dürfen sich unterscheiden, da die Compiler intern unterschiedlich arbeiten. Unter vergleichbaren Bedingungen muss das definierte beobachtbare Verhalten und damit auch die verglichene Ausgabe übereinstimmen.

6 Erzeugten Maschinencode auf dem ESP32-C3 testen

6.1 Testgegenstand und erwartetes Verhalten

Ein Ausführungstest untersucht, ob der erzeugte Code bei festgelegten Eingaben das erwartete Ergebnis liefert, wofür das übersetzte und gelinkte Programm ausgeführt wird. Gegenüber einem Vergleich des Assemblytexts werden damit auch die Verknüpfung mit der Laufzeitumgebung und die Ausführung auf dem Zielgerät berücksichtigt.

Als Beispiel dient die Funktion `add` aus dem LLVM-Test `alu32.ll`. Der folgende Ausschnitt zeigt die Funktion ohne ihre FileCheck-Direktiven [11].

```
define i32 @add(i32 %a, i32 %b) nounwind {
  %1 = add i32 %a, %b
  ret i32 %1
}
```

Listing 12: IR-Funktion aus alu32.ll, FileCheck-Direktiven weggelassen

Die Funktion erhält zwei 32-Bit-Integer-Parameter `%a` und `%b`, addiert sie und gibt das in `%1` berechnete Ergebnis zurück. Das Attribut `nounwind` schließt das Weiterreichen von Ausnahmen an den Aufrufer aus [25].

Für die Eingaben 2 und 5 ist das Ergebnis 7 zu erwarten. Die dargestellte LLVM-Addition berechnet das Ergebnis modulo 2^{32}. Für die Bitmuster `0xffffffff` und 1 ergibt sich daher 0 [25, Abschnitt *add Instruction*]. Diese Sollwerte werden aus der festgelegten Semantik abgeleitet und anschließend mit den tatsächlich ermittelten Ergebnissen verglichen.

6.2 Ablauf und Anbindung des erzeugten Codes

Der erzeugte Assemblycode wird zu einer Objektdatei assembliert. Diese wird mit dem Testprogramm und der benötigten Laufzeitumgebung gelinkt. Das ausführbare Programm wird anschließend auf den ESP32-C3 übertragen und gestartet.

Der Test muss die vom untersuchten Compiler erzeugte Funktion mit den festgelegten Eingaben aufrufen und ihren Rückgabewert auslesen können. Dafür müssen das Funktionssymbol, die Parameter- und Rückgabetypen sowie die verwendete ABI zwischen dem erzeugten Code und dem aufrufenden Testprogramm übereinstimmen. Die Grundlagen dieser Schnittstellen werden in den vorhergehenden Seminarthemen behandelt.

Für einen nachvollziehbaren Testlauf müssen die ausgeführte Programmversion, die verwendeten Compileroptionen, die Eingaben sowie die erwarteten und beobachteten Ergebnisse festgehalten werden. Dadurch bleibt erkennbar, welche Übersetzung tatsächlich geprüft wurde.

6.3 Beispiel für den Testaufbau mit Unity

ESP-IDF stellt für Tests auf dem ESP32-C3 eine Testanwendung mit dem C-Testframework Unity bereit [26]. Das folgende Originalbeispiel aus `test_mean.c` zeigt, wie Eingabe, Funktionsaufruf und Ergebnisvergleich in einem Test zusammengeführt werden. Es prüft eine Mittelwertfunktion in C [27].

```
TEST_CASE("Mean of a test vector", "[mean]")
{
    const int v[] = {1, 3, 5, 7, 9};
    TEST_ASSERT_EQUAL(5, testable_mean(v, countof(v)));
}
```

Listing 13: Originaltest aus dem ESP-IDF-Beispiel test_mean.c

`TEST_CASE` registriert den benannten Test. `countof` ist ein im Originalbeispiel definiertes Makro zur Bestimmung der Elementanzahl. Der erwartete Mittelwert der fünf Werte beträgt 5. `TEST_ASSERT_EQUAL` vergleicht diesen Sollwert mit dem Rückgabewert von `testable_mean` [27].

Das Beispiel veranschaulicht den Testaufbau. Um damit einen Codegenerator zu prüfen, muss die aufgerufene Implementierung aus der zu untersuchenden Übersetzung stammen. Beim Additionsbeispiel würde der Test die erzeugte Funktion mit den vorgesehenen Argumenten aufrufen und ihren Rückgabewert mit dem jeweiligen Sollwert vergleichen.

6.4 Auswertung und Aussagegrenzen

Für den beschriebenen Funktionstest müssen der erwartete Rückgabewert und ein regulärer Abschluss des Aufrufs vorliegen. Ein falscher Wert, ein Absturz und ein ausbleibendes Ergebnis sind unterschiedliche Fehlerfälle. Eine festgelegte Zeitgrenze ermöglicht es, auch einen nicht abgeschlossenen Aufruf als Fehler zu erfassen [24].

Eine Abweichung muss anschließend eingegrenzt werden. Ihre Ursache kann im erzeugten Code, in einer fehlerhaften Funktionsanbindung oder im Testaufbau liegen. Der beobachtete Fehler beweist deshalb noch nicht, dass allein der Codegenerator verantwortlich ist.

Ein bestandener Test bestätigt das erwartete Verhalten für die geprüften Eingaben auf dem verwendeten Gerät und mit der eingesetzten Programmversion. Eine allgemeine Korrektheit des Compilers lässt sich daraus nicht ableiten.

A Ergänzende Fundstellen

Die folgenden Fundstellen ergänzen die Literaturangaben. LLVM- und Clang-Fundstellen in diesem Anhang beziehen sich auf `llvmorg-20.1.0`, das ESP-IDF-Beispiel auf `v5.5`. Die Verweise führen zu den Onlinequellen. Codeausschnitte im Text sind Originalbeispiele oder gekürzte Auszüge; zusätzliche Indexberechnungen und Sollwerte erläutern die Beispiele.

1) *Clang CFE Internals Manual*: *The Diagnostics Subsystem*, *Diagnostic Wording*, *The Token class*, *SourceRange and CharSourceRange*, *Recovery AST*, *How to add an expression or statement* und *Specifying Diagnostics* [2].
2) `SourceLocation.h`: Kommentar zu `CharSourceRange` ab Zeile 248. Ergänzende API-Dokumentation der Bereichsarten [3].
3) `LexerTest.cpp`: `FindNextToken`, Zeilen 604–621. Eingabe und erwartete Tokenfolge stehen im selben Test [28].
4) `AsmParserTest.cpp`: `TypeAndConstantValueParsing`, ab Zeile 70. Der Testaufbau enthält die Variablen `M`, `Error` und `V` [29].
5) `MachineInstrTest.cpp`: `DebugLocPrinting`, ab Zeile 202. Vor den ausgegebenen Prüfzeilen stehen der Aufbau der Instruktion und ihre Quelltextzuordnung [30].
6) `alu32.ll`: `add` ab Zeile 164 und `sub` ab Zeile 178. Die Datei enthält auch RV64I-Erwartungen; im Text werden RV32I und RV64I gegenübergestellt [11].
7) `2002-05-02-CastTest1.c` und die zugehörige `.reference_output`: vollständiger Golden-Testfall. *LLVM test-suite Guide*, Abschnitt *Structure*, erklärt die Referenzausgaben [14, 13].
8) *LLVM Testing Infrastructure Guide*: *Unit tests*, *Regression tests* und *Regression test structure* [31].
9) *FileCheck-Dokumentation*: *Tutorial*, *CHECK-NEXT*, *CHECK-LABEL* und *strict-whitespace* [12].
10) *Csmith README*: *Use Csmith*, mit den übernommenen GCC-/Clang-Befehlen und dem Hinweis auf Zeitgrenzen [24].
11) *LLVM Language Reference Manual*: *Well-Formedness* und *add Instruction*; *CompCert-Handbuch*: Abschnitte 1.2 und 1.3 zu semantischer Erhaltung und Beweisumfang [25, 32].
12) *ESP-IDF Unit Testing*: *Normal Test Cases*, *Building Unit Test App*, *Running Unit Tests* und *Multi-stage Test Cases*. *test_mean.c*: Test `Mean of a test vector` [26, 27].
13) *RFC 3629*: Abschnitt 3 zur UTF-8-Codierung. *C11-Entwurf N1570*: Abschnitte 6.3.1.1 und 6.5.2.2 zur Integer-Promotion und Argumentumwandlung sowie 6.5.5 und 6.5.6 zur Operatorbindung [4, 15].

Die Originaltests benötigen teilweise die vollständige LLVM-/Clang- oder ESP-IDF-Testumgebung. Sie erläutern hier die Verfahren und wurden für diese Ausarbeitung nicht als vollständige Testsammlung ausgeführt.

Literatur

[1] Rust Project. *UI Tests*. 2026. URL: https://rustc-dev-guide.rust-lang.org/tests/ui.html (besucht am 31. 10. 2026).

[2] LLVM Project. *Clang CFE Internals Manual*. Version 20.1.0. Abschnitte Diagnostics, SourceRange, Testing und Specifying Diagnostics. URL: https://releases.llvm.org/20.1.0/tools/clang/docs/InternalsManual.html (besucht am 29. 10. 2026).

[3] LLVM Project. *SourceLocation.h: CharSourceRange*. Stand llvmorg-20.1.0, API-Dokumentation ab Zeile 248. URL: https://github.com/llvm/llvm-project/blob/llvmorg-20.1.0/clang/include/clang/Basic/SourceLocation.h (besucht am 30. 10. 2026).

[4] F. Yergeau. *RFC 3629: UTF-8, a transformation format of ISO 10646*. November 2003, Abschnitt 3. URL: https://www.rfc-editor.org/rfc/rfc3629.html (besucht am 30. 10. 2026).

[5] Gerard Meszaros. *xUnit Test Patterns: Refactoring Test Code*. Boston: Addison-Wesley, 2007. ISBN: 9780131495050.

[6] Free Software Foundation. *The GNU C Preprocessor Internals*. GNU Compiler Collection Documentation. URL: https://gcc.gnu.org/onlinedocs/cppinternals.pdf (besucht am 01. 11. 2026).

[7] LLVM Project. *Clang Lexer Unit Tests (LexerTest.cpp)*. 2026. URL: https://github.com/llvm/llvm-project/blob/main/clang/unittests/Lex/LexerTest.cpp (besucht am 30. 10. 2026).

[8] Python Software Foundation. *test_tokenize.py*. 2026. URL: https://github.com/python/cpython/blob/9c0cfb141c236c902954841bb6cca407f86e1eeb/Lib/test/test_tokenize.py#L2010-L2044 (besucht am 01. 11. 2026).

[9] Python Software Foundation. *tokenize.py*. 2026. URL: https://github.com/python/cpython/blob/main/Lib/tokenize.py#L48-L49 (besucht am 01. 11. 2026).

[10] Rust Project. *edition-keywords-2018-2015-parsing.rs*. 2026. URL: https://github.com/rust-lang/rust/blob/main/tests/ui/editions/edition-keywords-2018-2015-parsing.rs (besucht am 01. 11. 2026).

[11] LLVM Project. *RV32I-Backendtests in alu32.ll*. Stand llvmorg-20.1.0, Funktionen add und sub. URL: https://github.com/llvm/llvm-project/blob/llvmorg-20.1.0/llvm/test/CodeGen/RISCV/alu32.ll (besucht am 29. 10. 2026).

[12] LLVM Project. *FileCheck: Flexible pattern matching file verifier*. Version 20.1.0, Tutorial. URL: https://releases.llvm.org/20.1.0/docs/CommandGuide/FileCheck.html (besucht am 29. 10. 2026).

[13] LLVM Project. *test-suite Guide*. Version 20.1.0, Abschnitt Structure. URL: https://releases.llvm.org/20.1.0/docs/TestSuiteGuide.html (besucht am 29. 10. 2026).

[14] LLVM Project. *2002-05-02-CastTest1: C-Test und Referenzausgabe*. Referenzdatei im selben Ordner: 2002-05-02-CastTest1.reference_output. URL: https://github.com/llvm/llvm-test-suite/blob/llvmorg-20.1.0/SingleSource/UnitTests/2002-05-02-CastTest1.c (besucht am 29. 10. 2026).

[15] ISO/IEC JTC 1/SC 22/WG 14. *Programming Languages — C*. Committee Draft N1570. 12. April 2011, Abschnitte 5.2.4.2.2, 6.3.1.1, 6.5.2.2, 6.5.5 und 6.5.6. ISO/IEC, Apr. 2011. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf.

[16] Luiz Paulo Grafetti Terres und Samuel da Silva Feitosa. „Differential Fuzzing Go Compilers using LLMs: A Methodological Proposal“. In: *Anais da Escola Regional de Engenharia de Software (ERES)*. Sociedade Brasileira de Computação, 29. Okt. 2025, S. 50–59. DOI: 10.5753/eres.2025.16857. URL: https://sol.sbc.org.br/index.php/eres/article/view/40378 (besucht am 30. 10. 2026).

[17] Rana Elnaggar, Brian Delgado und Jason M. Fung. *RAG-Based Fuzzing of Cross-Architecture Compilers*. Version 1. 11. Apr. 2025. arXiv: 2504.08967 [cs.CR]. URL: https://arxiv.org/abs/2504.08967v1 (besucht am 30. 10. 2026).

[18] cppreference. *Arithmetic Operators*. URL: https://cppreference.com/c/language/operator_arithmetic (besucht am 01. 11. 2026).

[19] cppreference. *Order of Evaluation*. URL: https://en.cppreference.com/cpp/language/eval_order (besucht am 01. 11. 2026).

[20] cppreference. *Undefined Behavior*. URL: https://en.cppreference.com/c/language/behavior (besucht am 01. 11. 2026).

[21] Free Software Foundation. *Options That Control Optimization*. GNU Compiler Collection (GCC) Documentation. URL: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html (besucht am 01. 11. 2026).

[22] H. J. Lu u. a. *System V Application Binary Interface: AMD64 Architecture Processor Supplement, with LP64 and ILP32 Programming Models*. Version 1.0, 28 January 2018, Abschnitt 3.1.2. Jan. 2018. URL: https://github.com/hjl-tools/x86-psABI/wiki/x86-64-psABI-1.0.pdf.

[23] Arm Limited. *Procedure Call Standard for the Arm Architecture*. Abschnitte 5.1, 5.2 und 8.1.1; abgerufen am 1. November 2026. URL: https://github.com/ARM-software/abi-aa/blob/main/aapcs32/aapcs32.rst.

[24] Csmith Project. *Csmith, a random generator of C programs*. 2026. URL: https://github.com/csmith-project/csmith/blob/master/README.md (besucht am 30. 10. 2026).

[25] LLVM Project. *LLVM Language Reference Manual*. Version 20.1.0, Abschnitte Well-Formedness und add Instruction. URL: https://releases.llvm.org/20.1.0/docs/LangRef.html (besucht am 30. 10. 2026).

[26] Espressif Systems. *ESP-IDF: Unit Testing in ESP32-C3*. Version 5.5, Abschnitt Normal Test Cases. URL: https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32c3/api-guides/unit-tests.html (besucht am 29. 10. 2026).

[27] Espressif Systems. *ESP-IDF: test_mean.c*. Stand v5.5, Test Mean of a test vector. URL: https://github.com/espressif/esp-idf/blob/v5.5/examples/system/unit_test/components/testable/test/test_mean.c (besucht am 30. 10. 2026).

[28] LLVM Project. *Clang LexerTest.cpp: FindNextToken*. Stand llvmorg-20.1.0, Zeilen 604–621. URL: https://github.com/llvm/llvm-project/blob/llvmorg-20.1.0/clang/unittests/Lex/LexerTest.cpp (besucht am 29. 10. 2026).

[29] LLVM Project. *AsmParserTest.cpp: TypeAndConstantValueParsing*. Stand llvmorg-20.1.0, Test ab Zeile 70. URL: https://github.com/llvm/llvm-project/blob/llvmorg-20.1.0/llvm/unittests/AsmParser/AsmParserTest.cpp (besucht am 30. 10. 2026).

[30] LLVM Project. *MachineInstrTest.cpp: DebugLocPrinting*. Stand llvmorg-20.1.0, Test ab Zeile 202. URL: https://github.com/llvm/llvm-project/blob/llvmorg-20.1.0/llvm/unittests/CodeGen/MachineInstrTest.cpp (besucht am 30. 10. 2026).

[31] LLVM Project. *LLVM Testing Infrastructure Guide*. Version 20.1.0, Abschnitte Unit tests und Regression tests. URL: https://releases.llvm.org/20.1.0/docs/TestingGuide.html (besucht am 29. 10. 2026).

[32] CompCert Project. *CompCert C: a trustworthy compiler*. Kapitel 1, insbesondere 1.2 und 1.3. URL: https://compcert.org/man/manual001.html (besucht am 30. 10. 2026).